

Learning Based Grasping with 3D Vision

Sahen Juneja

Johns Hopkins University

3400 N. Charles Street, Baltimore, MD 21218

sjuneja3@jh.edu

Abstract

The goal of this project is to explore the potential of 3D Vision in machine learning models for grasping based tasks. To achieve this a custom simulation environment was build on Pandas-Gym, combining the YCB dataset to create a pick and place task. Pybullet and Open3D were used to construct and clean point clouds. Down sampled point clouds, observations from PickAndPlace-v3, and a switch were trained on multilayered perceptrons which fed into a policy head and outputted to a soft actor critic which returned results in the action space. Training took more time then anticipated and even with optimizations a successful model was not able to be generated. This paper outlines methods in which this issue could be resolved as well as other extensions in future work.

1. Introduction

“Spatial reasoning is fundamental to all intellectual processes” [10]. Understanding the environment is a key aspect of what allows humans to act in an intelligent manner. For decades researchers have worked on allowing robots to perceive the world around them as sharply as humans. Robot perception evolved from photo-cells to detect changes in light to modern methods such as learning-based vision to gain a nuanced understanding of the environment.

Generalization remains one of the central challenges in vision-based robotic grasping. Robot grasping is a task that consists of two parts, “perception and planning” [9]. In Du et al.’s survey of vision based grasping algorithms, they concluded the steps were 1) object localization 2) object pose estimation and 3) grasp estimation [2]. Du et al. specify that object localization does not include gaining a further understanding of properties of the object such as classification but rather through correspondence and a voting based algorithm a grasp method is chosen. Picking grasp through simple correspondence restricts the generalization of grasping as if a object is not in a grasping algorithm’s dataset then no grasp is able to be generated. Vision-Language Ac-

tion (VLA) models are able to generate actions for objects that are not in it’s trained dataset but generalization is not a solved problem [3, 10].

3D object reconstruction from 2D images is a method that can extrapolate physical features from images. While shadows, edges, and other features can imply good regions to grasp an object, 3D representations have additional geometric ways to express graspability. This project seeks to explore 3D representations of objects for grasping. Through creating a custom Pandas-Gym environment and implementing object reconstruction a 3D-vision learning based model was created to study if 3D vision holds advantages in grasping tasks over 2D vision.

2. Related Work

3D reconstruction is a problem for which a variety of solutions exist. SAM 3D is a reconstruction algorithm which takes a single image and processes it through two diffusion transformers [8]. The first creates an estimation of the shape and pose while the second refines those qualities and defines texture. Gaussian Splatting is another learning based algorithm in which 3D Gaussian are used to approximate objects in a scene [5]. Initial 3D Gaussians are generated, those Gaussians are then split, pruned, and otherwise changed to better approximate the scene, and finally they are projected into 2D for faster rendering.

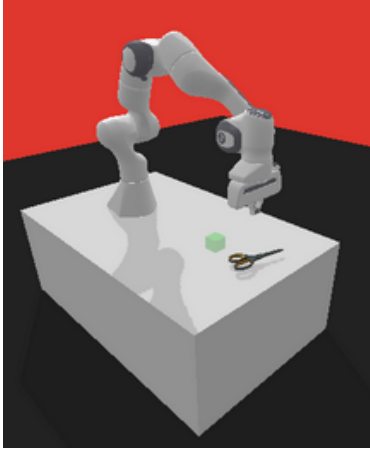
Point Cloud based grasping exists in modern research in a limited form. While point clouds in an end-to-end grasp algorithm is present, there are several challenges. A main struggle is with localizing the target object pose [9]. This project seeks to fix this issue by fusing observations with the Point Cloud.

3. Methodology

3.1. Building Sim

This project starts with building a custom simulation. There are several environments on which a simulation could be built on such as Rviz, Mujoco, but ultimately Pandas-Gym was selected for 2 reasons. The first of which is its prebuilt

environments similarity to the task. PandasPickAndPlace-v3 is a default task in which a Franka Emika Panda manipulator picks a cube in its parallel gripper end effector and brings it to a goal [4]. Because part of this experiment is to test the generalization of 3D vision based grasping, the green cube was swapped for a random object from the Yale-CMU-Berkeley (YCB) benchmark dataset, a dataset of .obj files for dozens of household objects [1]. Second, Pandas-Gym is an environment built for machine learning. It is a light weight simulation environment which has pre-existing integrations with packages such as RL Baselines3 Zoo and provides rewards and observations every timestep.



(a) Custom Pick And Place Environment



(b) Example Object from YCB dataset

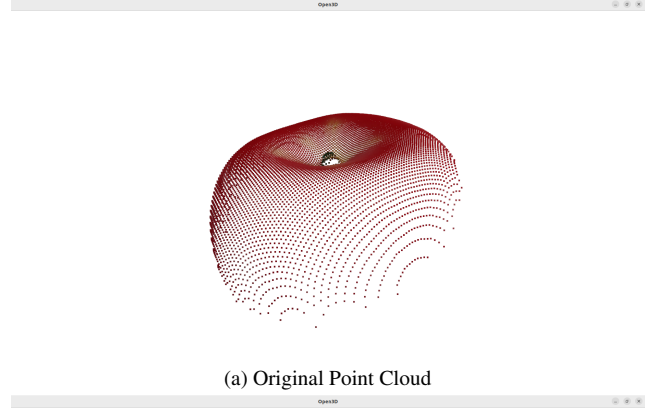
Figure 1. Point cloud preprocessing pipeline.

3.2. Reconstruction

Using Pandas-Gym’s underlying physics simulator, PyBullet, the a eye-in-hand camera was created for reconstruction. By indexing robot links for the end effector, the pose for the gripper was gained. Using PyBullet’s `computeViewMatrix()` and

`computeProjectionMatrixFOV()`, the parameters for rendering were defined and `getCameraImage()` rendered RGBD images to act as a simulated eye-in-hand camera. To reduce noise in the image for stable network training, points in the RGBD images were masked for the object ID to remove the background in a pseudo-object detection and segmentation.

Open3D was used to turn RGBD images into cleaned point clouds. With known camera intrinsics the RGBD images were converted to point clouds with `create_from_color_and_depth()` as seen in Figure 2a. The points are down sampled with `voxel_down_sample()` to preserve shape and then `remove_statistical_outlier()` to remove outlier points. To further improve stability in training, the point cloud size is normalized and centered around the origin. Finally the number of points are up or downsized to match the neural network input size, which in this study is 1024, as shown in Figure 2b.



(b) Downsampled Point Cloud

Figure 2. Point cloud preprocessing pipeline.

3.3. Policy Architecture and Training

The control policy is implemented as a custom feature extractor. The policy was trained on observations which consisted of the cleaned point cloud, the state, and a point cloud switch. A multi-layer perceptron (MLP) network

with two hidden linear layers and a ReLU activation function extracted features from the point cloud. Similarly a MLP with one hidden linear layer and a ReLU activation function learned embeddings for the state, which consisted of "observation", "achieved_goal", and "desired_goal" from Pandas-Gym PickAndPlace-v3 task. The policy implemented the output of the two MLP and the point cloud switch, which was a 1 or 0 depending if the point cloud was empty, through a linear and ReLU layer returning a (batch_size, 256) vector. The idea behind this was to train the model to learn how to grasp with the point cloud, how to approach the object with the states, and whether or not to trust the point cloud based on the switch. Finally this policy was then trained with a soft actor critic in RL Baselines3 Zoo to produce an action within the specified action space.

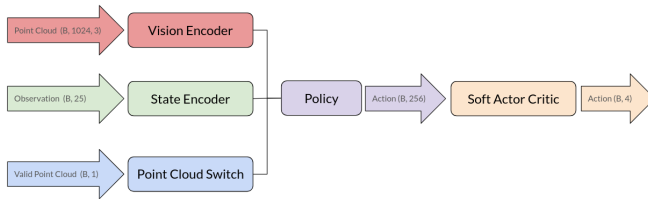


Figure 3. Model Architecture

3.4. Experimental Setup

The task for the robot was to pick up an object and bring it to a goal. The goal and object were randomly picked from Pandas PickAndPlace-v3 sample space. The object was a random URDF from the YCB dataset. The task terminated successfully when the robot brought the object within a threshold distance of the goal, else if the number of steps were exceeded, the task terminated as a failure. The rewards are defined as the default for PickAndPlace-v3.

3.5. Additional Explorations

It was originally planned to train models with just a state feature extractor and SAC to serve as a baseline and evaluation method. The reason this didn't happen is explained in results. Other ideas for analyzing the effectiveness of 3D vision based gripping included comparing against other state of the art parallel gripper implementations such as OpenVLA [6]. Additional planned extensions of the project included using pretrained point cloud feature extractors such as PointNet++ or training with DAgger and an expert [7]. These extensions could be done in future work.

4. Discussion

4.1. Results

The results of this study are inconclusive. One model was trained using SAC and a custom policy with 10,000 steps of training producing a success rate of 0%. Compared to similar tasks such as the default Pandas-Gym PickAndPlace-v3, it makes sense that training only 10,000 steps would result in a model that hasn't learned any of the system dynamics or how to achieve the goal as those tasks often require millions of steps and tuned hyperparameter to train properly. While it was initially the goal to train the model on higher timesteps and use Optuna for automated hyperparameter tuning, the trial with only 10,000 steps took four hours of training. After optimizing computational speed by reducing rendering and point cloud processing, another model which never completed training trained for 12 hours and only processed 25,000 steps out of its 500,000 goal as seen in 4. The bar showed no progress after 12 hours and warnings such as "This system does not have apparently enough memory to store the complete replay buffer 24.81GB>5.96GB" showed up, indicating that stronger hardware would be required to train this policy. Overall while this approach may have worked, more optimization and potentially upgrading hardware would be required.

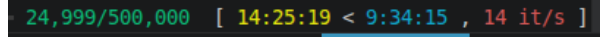


Figure 4. Training Progress Bar: Green is steps completed, Yellow is time progressed, Blue is estimated time to completion, Red is steps trained per second

4.2. Future Works

As discussed in the methodology section, the creating baselines with only a state feature extractor as a policy or using OpenVLA would allow a proper evaluation of the effectiveness of this method. This however hinges on being able to train a model with an adequate training length and proper hyperparameter adjusting which hinge on decreasing training time or increasing hardware capabilities. Training time could be decreased through using pretrained 3D vision models such as PointNet++ or using quicker training methods like DAgger. Hardware could be improved through collecting data locally and training models on cloud-based virtual machines such as Google Colab. If this training times were solved, this project could continue to be extended in a number of ways. The first of which is expanding the generalization of the task. In addition to the training and evaluation datasets increasing in size, the task could change from picking objects on the floor to picking objects in cluttered spaces, requiring the gripper to learn to change orientation and understand a scene rather than an object. The point cloud could be continuously added to rather than remaining

static, incentivizing the robot to explore and gain a multi-view understanding of the object before proceeding.

References

- [1] Berk Calli, Aaron Walsman, Arjun Singh, Siddhartha Srinivasa, Pieter Abbeel, and Aaron M. Dollar. Ycb object and model set, 2015. Yale-CMU-Berkeley object models for robotic manipulation benchmarking. [2](#)
- [2] Guoguang Du, Kai Wang, Shijian Lian, and Kai Zhao. Vision-based robotic grasping from object localization, object pose estimation to grasp estimation for parallel grippers: A review. *Artificial Intelligence Review*, 54(3):1677–1734, 2021. [1](#)
- [3] Dean Fortier, Timothy Adamson, Tess Hellebrekers, Teresa LaScala, Kofi Ennin, Michael Murray, Andrey Kolobov, and Galen Mullins. Benchmarking affordance generalization with busybox. *arXiv preprint arXiv:2602.05441*, 2026. [1](#)
- [4] Quentin Gallouédec, Nicolas Cazin, Emmanuel Dellandréa, and Liming Chen. panda-gym: Open-source goal-conditioned environments for robotic learning. *4th Robot Learning Workshop: Self-Supervised and Lifelong Learning at NeurIPS*, 2021. [2](#)
- [5] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4):1–14, 2023. [1](#)
- [6] Moo Jin Kim, Fei Xia, Ted Xiao, Zichen Wang, Yu-Wei Chao, Jimmy Xu, Xinyi Wang, Xingyu Zhang, Brian Ichter, Danny Driess, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024. [3](#)
- [7] Charles Ruizhongtai Qi, Li Yi, Hao Su, and Leonidas J. Guibas. Pointnet++: Deep hierarchical feature learning on point sets in a metric space. *arXiv preprint arXiv:1706.02413*, 2017. [3](#)
- [8] SAM 3D Team, Xingyu Chen, Fu-Jen Chu, Pierre Gleize, Kevin J. Liang, Alexander Sax, Hao Tang, Weiyao Wang, Michelle Guo, Thibaut Hardin, Xiang Li, Aohan Lin, Jiawei Liu, Ziqi Ma, Anushka Sagar, Bowen Song, Xiaodong Wang, Jianing Yang, Bowen Zhang, Piotr Dollár, Georgia Gkioxari, Matt Feiszli, and Jitendra Malik. Sam 3d: 3dfy anything in images. *arXiv preprint arXiv:2511.16624*, 2025. [1](#)
- [9] Andreas ten Pas, Marcus Gualtieri, Kate Saenko, and Robert Platt. Grasp pose detection in point clouds. *The International Journal of Robotics Research*, 36(13-14):1455–1473, 2017. [1](#)
- [10] Wentao Yuan, Jiafei Duan, Valts Blukis, Wilbert Pumacay, Ranjay Krishna, Adithyavairavan Murali, Arsalan Mousavian, and Dieter Fox. Robopoint: A vision-language model for spatial affordance prediction for robotics. *arXiv preprint arXiv:2406.10721*, 2024. [1](#)